可变参数

“可变参数”（variadic）指一个函数可以接收任意数量的参数。在 Python 中，使用 `*args` 和 `**kwargs` 来实现这一功能。

- `*args`：将多个位置参数打包成一个元组（tuple）。
- `**kwargs`：将多个关键字参数打包成一个字典（dict）。

示例 1：计算任意数量数字的平均值

```
def average(*numbers):
    if not numbers:
        return 0
    return sum(numbers) / len(numbers)

print(average(1, 2, 3))  # 输出 2.0
print(average(10, 20))  # 输出 15.0
```

示例 2：打印任意信息

```
def print_info(*messages, **options):
    sep = options.get('sep', ' ')
    end = options.get('end', '\n')
    for msg in messages:
        print(msg, sep=sep, end=end)

print_info("Hello", "World")  # 输出 Hello World
print_info("Hello", "World", sep="-")  # 输出 Hello-World
```

解析

- `*numbers`：将所有位置参数打包成一个元组 `(1, 2, 3)`。
- `**options`：将所有关键字参数打包成一个字典 `{'sep': '-'}`。
- `options.get('sep', ' ')`：从字典中获取 `sep` 的值，默认值为 `' '`（空格）。
- `options.get('end', '\n')`：从字典中获取 `end` 的值，默认值为 `'\n'`（换行）。
- `for msg in messages`：遍历元组中的每个参数。
- `print(msg, sep=sep, end=end)`：使用指定的 `sep` 和 `end` 参数打印。

解包

解包（unpacking）是将一个可迭代对象（如列表、元组、字典）的内容展开，分别赋值给多个变量的过程。

示例 1：列表解包

```
numbers = [1, 2, 3]
a, b, c = numbers
print(a, b, c)  # 输出 1 2 3
```

示例 2：字典解包

```
def greet(name, age):
    print(f"Hello, {name}! You are {age} years old.")

info = {'name': 'Alice', 'age': 30}
greet(**info)  # 输出 Hello, Alice! You are 30 years old.
```

解析

- `a, b, c = numbers`：将列表 `[1, 2, 3]` 解包，分别赋值给 `a`、`b`、`c`。
- `greet(**info)`：将字典 `info` 解包，传递给函数 `greet` 的参数 `name` 和 `age`。

生成器

生成器（generator）是一种特殊的迭代器，使用 `yield` 关键字来逐个生成值，而不一次性将所有值存入内存。

示例：斐波那契数列

```
def fibonacci(limit):
    a, b = 0, 1
    while a < limit:
        yield a
        a, b = b, a + b

for num in fibonacci(10):
    print(num, end=" ")  # 输出 0 1 1 2 3 5 8
```

解析

- `yield a`：每次生成一个值，暂停函数执行，保存当前状态。
- `a, b = b, a + b`：计算下一个斐波那契数。
- `for num in fibonacci(10)`：遍历生成器，逐个获取值。

装饰器

装饰器（decorator）是一种用于修改函数行为的工具，通常用于日志记录、性能测试、权限检查等。

示例：计时装饰器

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} 耗时 {end - start:.2f} 秒")
        return result
    return wrapper

@timer
def slow_function():
    time.sleep(2)

slow_function()  # 输出 slow_function 耗时 2.00 秒
```

解析

- `@timer`：等价于 `slow_function = timer(slow_function)`。
- `*args, **kwargs`：保证装饰器能处理任意参数的函数。
- `func.__name__`：获取被装饰函数的名称。

上下文管理器

上下文管理器（context manager）用于管理资源的分配和释放，通常用于文件操作、数据库连接、网络连接等。

示例：文件操作

```
with open("example.txt", "w") as f:
    f.write("Hello, World!")
```

解析

- `with open("example.txt", "w") as f`：打开文件，自动分配资源。
- `f.write("Hello, World!")`：写入内容。
- 文件自动关闭，即使发生错误也会确保资源释放。

推荐学习

- 官方文档：https://docs.python.org/3/
- 实用教程：https://realpython.com/
- 练习平台：https://leetcode.com/

关注获客时间服务号
每日伊卫签到
每日一句 / 每日一题
连续签到 奖励积分 抽奖 / 兑换 / 排行榜